Engineering Contract & Technical Agreements

Project: Online Shop (MVP)
Roles: Admin, Trader, Customer, Courier
Stack (fixed):

- Backend: Java 17, Spring Boot, Spring Security, JPA/Hibernate
- Database: PostgreSQL
- Auth: JWT (access + refresh)
- Frontend Web: HTML, CSS, JavaScript, React
- Mobile: React Native
- Transport: REST over HTTPS
- Media: URL-based (no binary storage in MVP)

---

1. Team Structure & Responsibility Split

1.1 Backend Team (Main Platform) — 3 Developers

Owns:

- Core domain: Users, Roles, Products, Categories, Orders
- Admin Panel APIs
- Customer Platform APIs
- Authentication & RBAC
- Central inventory & pricing (source of truth)
- Courier task assignment logic
- Audit logs

Does NOT own:

- Trader CMS
- Courier mobile

1.2 Backend Developer (Trader CMS) — 1 Developer

Owns:

- Trader CMS backend
- Product sync logic (pull from Admin API)
- Local trader product storage
- Trader order views & notes
- API key management (client-side usage)

Constraints:

- Must use Admin APIs as external dependency
- No business logic override (price, stock, categories are read-only)

1.3 Frontend Web — 2 Developers

Owns:

- Admin Panel (React)
- Customer Platform (React)
- Auth flows for Admin / Customer
- UI state handling based on API contracts

1.4 Mobile Team (Courier App) — 2 Developers

Owns:

- Courier mobile application
- Offline-first behavior
- Push notifications
- Evidence upload

Does NOT own:

- Order lifecycle logic
- Assignment logic

---

2. Global Technical Rules (Binding)

2.1 API Standards

- REST, JSON only
- Versioning: `/api/v1/...`
- HTTP status codes strictly enforced
- Idempotent POSTs where status can be retried
- UTC timestamps (ISO-8601)

2.2 Authentication

- JWT Access Token (short-lived)
- JWT Refresh Token (long-lived)
- Tokens passed via `Authorization: Bearer <token>`
- No cookies (mobile-first compatible)

2.3 Authorization

RBAC enforced at **controller level**:

- `ADMIN`
- `TRADER`
- `CUSTOMER`
- `COURIER`

Unauthorized access → `403 Forbidden`
Unauthenticated → `401 Unauthorized`

---

3. Identity & Account Rules (Hard Constraints)

Role	Created By	Can Register	Can Login
Admin	Manual (system)	❌	✅

Trader	Self-registration	✅	✅ (after approval)
Customer	Trader	❌	✅
Courier	Admin	❌	✅

No password reset in MVP.

---

4. Core API Contracts (Cross-Team)

4.1 Auth (Shared)

```
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
```

Response:

```
{
  "accessToken": "jwt",
  "refreshToken": "jwt",
  "role": "TRADER",
  "userId": 123
}
```

---

5. Admin APIs (Main Backend)

Users

```
POST   /api/v1/admin/couriers
GET    /api/v1/admin/couriers
PUT    /api/v1/admin/couriers/{id}
DELETE /api/v1/admin/couriers/{id}

GET    /api/v1/admin/traders/pending
POST   /api/v1/admin/traders/{id}/approve
```

```
POST   /api/v1/admin/traders/{id}/reject
```

Catalog (Source of Truth)

```
POST /api/v1/admin/categories
GET  /api/v1/admin/categories

POST /api/v1/admin/products
GET  /api/v1/admin/products
GET  /api/v1/admin/products/{id}
```

Fields controlled ONLY by Admin:

- price
- central stock
- categories

Orders

```
GET /api/v1/admin/orders
PUT /api/v1/admin/orders/{id}/status
```

Admin can override any status.

---

6. Trader CMS ↔ Admin Contract (Critical)

6.1 Authentication

Trader CMS uses:

- JWT (login)
- API Key (issued after admin approval)

Every sync request MUST include:

```
Authorization: Bearer <access_token>
X-API-KEY: <trader_api_key>
```

6.2 Catalog Sync (Read-Only)

```
GET /api/v1/admin/sync/products?since=&page=
```

Response:

```
{
  "products": [
    {
      "sourceId": 12,
      "title": "Product",
      "price": 100,
      "centralStock": 20,
      "category": "Electronics",
      "version": "hash"
    }
  ]
}
```

Trader CMS MUST:

- Store products locally
- Never modify price or central stock
- Track `sourceId + version`

---

7. Trader CMS Internal APIs

```
GET   /api/v1/trader/products
PATCH /api/v1/trader/products/{id}
```

Editable fields:

- local_description
- local_images
- visibility

---

8. Customer APIs (Main Backend)

```
GET  /api/v1/products
GET  /api/v1/products/{id}
POST /api/v1/orders
GET  /api/v1/orders
```

Order creation:

- Locks price snapshot
- Decreases central stock
- Assigns trader

---

9. Courier APIs (Main Backend)

Tasks

```
GET  /api/v1/couriers/{id}/tasks
POST /api/v1/tasks/{id}/accept
POST /api/v1/tasks/{id}/status
POST /api/v1/tasks/{id}/evidence
```

Allowed statuses:

```
ASSIGNED → ACCEPTED → PICKED_UP → IN_TRANSIT → DELIVERED
                   ↘ FAILED
```

Courier:

- Cannot edit orders

- Cannot see other couriers' tasks

---

10. Offline & Mobile Rules (Courier App)

- Cache assigned tasks locally
- Queue status updates & evidence
- Retry on reconnect
- All actions timestamped
- Idempotent status updates required

---

11. Data Ownership Rules (Very Important)

Data	Owner
Price	Admin
Central stock	Admin
Category	Admin
Local images	Trader
Local description	Trader
Order status (delivery)	Courier
Order lifecycle	System

Violations = backend bug.

---

12. Non-Functional Requirements

- Stateless backend
- Avg response < 500ms (local)
- PostgreSQL transactions for orders
- Soft delete everywhere
- Audit logs for:

    - status changes
    - auth events
    - sync actions

---

13. Definition of Done (All Teams)

- API matches contract
- Role isolation verified
- No hardcoded IDs
- No cross-role data leaks
- Happy-path + negative tests pass
- Swagger/OpenAPI updated